

- WHU OS Lab 4 实验报告：特权级转换与首个用户态进程创建
 - 一、实验目的
 - 二、实验原理
 - 2.1 RISC-V 特权级切换
 - 2.1.1 U-mode 到 S-mode 的切换
 - 2.1.2 S-mode 到 U-mode 的切换
 - 2.2 进程数据结构
 - 2.2.1 进程控制块 (proc_t)
 - 2.2.2 Trapframe 结构
 - 2.2.3 Context 结构
 - 2.3 用户地址空间布局
 - 2.4 Trampoline 机制
 - 三、实验内容
 - 3.1 内存布局配置
 - 3.1.1 include/memlayout.h - 地址空间定义
 - 3.1.2 kernel/kernel.ld - 链接脚本修改
 - 3.2 内核虚拟内存映射扩展
 - 3.2.1 kernel/mem/vmem.c - kvm_init() 修改
 - 3.3 进程管理实现
 - 3.3.1 kernel/proc/cpu.c - myproc()
 - 3.3.2 kernel/proc/proc.c - proc_pgtbl_init()
 - 3.3.3 kernel/proc/proc.c - proc_make_first()
 - 3.4 用户态 Trap 处理
 - 3.4.1 kernel/trap/trap_user.c - trap_user_handler()
 - 3.4.2 kernel/trap/trap_user.c - trap_user_return()
 - 3.5 用户程序编译
 - 3.5.1 user/initcode.c - 首个用户程序
 - 3.5.2 user/Makefile - 编译规则
 - 3.6 启动流程更新
 - 3.6.1 kernel/boot/main.c - 修改
 - 四、关键技术点
 - 4.1 全局数据初始化为 0
 - 4.2 位置无关代码
 - 4.3 页表切换的安全性
 - 4.4 上下文切换流程
 - 五、遇到的问题及解决方案
 - 5.1 链接错误：undefined reference to interrupt_info 和 exception_info

- [5.2 Makefile 未正确重新链接](#)
- [5.3 系统在 kvm_inithart\(\) 后卡住](#)
- [5.4 printf 格式化输出错误](#)
- [5.5 kvm_init\(\) 中 trampoline 映射的时机问题](#)
- [5.6 initcode 编译的地址问题](#)
- [5.7 调试输出的缓冲问题](#)
- 六、实验测试
 - [6.1 测试代码](#)
 - [6.2 运行结果](#)

WHU OS Lab 4 实验报告：特权级转换与首个用户态进程创建

一、实验目的

本实验旨在实现 RISC-V 操作系统从内核态到用户态的特权级切换机制，并创建首个用户态进程，主要包括：

1. 理解 RISC-V 特权级转换的原理和流程
2. 实现用户态进程的数据结构和内存布局
3. 实现用户态页表的创建和映射
4. 实现 Trampoline 机制用于特权级切换
5. 实现用户态 trap 处理流程
6. 创建并启动首个用户态进程 proczero
7. 理解上下文切换机制

二、实验原理

2.1 RISC-V 特权级切换

2.1.1 U-mode 到 S-mode 的切换

当用户态程序执行 trap（系统调用、异常或中断）时，硬件自动完成以下操作：

1. 如果是设备中断且 `sstatus.SIE = 0`，不进行切换
2. 通过置零 `SIE` 禁用中断
3. 将当前 `pc` 拷贝到 `sepc`
4. 保存当前特权级到 `sstatus.SPP`
5. 设置 `scause` 为 trap 原因
6. 设置当前特权级为 Supervisor
7. 将 `stvec` 拷贝到 `pc`，跳转到 trap 处理程序

注意: CPU 不会自动切换页表或栈，这些需要软件完成。

2.1.2 S-mode 到 U-mode 的切换

从内核态返回用户态时，需要手动设置：

1. 清除 `sstatus.SPP`，将其置为 0（表示返回 U-mode）
2. 设置 `sstatus.SPIE = 1`，启用用户态中断
3. 设置 `sepc` 为用户进程的 PC 值
4. 切换到用户进程的页表（写 `satp`）
5. 恢复用户态寄存器上下文
6. 执行 `sret` 指令

硬件在执行 `sret` 时自动完成：

- 从 `sepc` 恢复 `pc`
- 从 `sstatus` 恢复用户模式状态
- 将特权模式设置为用户模式

2.2 进程数据结构

2.2.1 进程控制块 (proc_t)

```
typedef struct proc {  
    int pid;                // 进程标识符  
    pgtbl_t pgtbl;          // 用户态页表  
    uint64 heap_top;        // 用户堆顶（字节为单位）  
    uint64 ystack_pages;    // 用户栈占用的页面数量  
    trapframe_t* tf;        // Trapframe（用户态/内核态切换时的寄存器保存区）  
    uint64 kstack;          // 内核栈的虚拟地址  
    context_t ctx;          // 内核态进程上下文  
} proc_t;
```

2.2.2 Trapframe 结构

Trapframe 用于在用户态和内核态切换时保存寄存器：

```
typedef struct trapframe {
    // 内核信息
    uint64 kernel_satp;    // 内核页表
    uint64 kernel_sp;      // 内核栈指针
    uint64 kernel_trap;    // trap 处理函数地址
    uint64 kernel_hartid;   // CPU ID

    // 用户态切换信息
    uint64 epc;             // 用户程序计数器

    // 通用寄存器 (31个, x0 固定为0)
    uint64 ra;
    uint64 sp;
    uint64 gp;
    uint64 tp;
    uint64 t0, t1, t2;
    uint64 s0, s1;
    uint64 a0, a1, a2, a3, a4, a5, a6, a7;
    uint64 s2, s3, s4, s5, s6, s7, s8, s9, s10, s11;
    uint64 t3, t4, t5, t6;
} trapframe_t;
```

2.2.3 Context 结构

Context 用于进程间的上下文切换：

```
typedef struct context {
    uint64 ra; // 返回地址
    uint64 sp; // 栈指针

    // 被调用者保存寄存器
    uint64 s0, s1, s2, s3, s4, s5, s6, s7, s8, s9, s10, s11;
} context_t;
```

Context 和 Trapframe 的区别：

- **Context:** 用于同一特权级内的进程切换（内核态进程之间）
- **Trapframe:** 用于不同特权级之间的切换（用户态 ↔ 内核态）

2.3 用户地址空间布局

高地址

TRAMPOLINE	最高页，跳板代码（用户态和内核态共享）
TRAPFRAME	Trapframe 页（用户态和内核态共享）
User Stack	用户栈（向下增长）
Heap	heap_top 堆（向上增长）
Code + Data	PGSIZE（0x1000） 代码和数据段
Empty	0x0 最低 4KB 不映射（捕获空指针）

低地址

2.4 Trampoline 机制

Trampoline（跳板）页是一个同时映射在用户页表和内核页表中的特殊页面：

- **位置:** 虚拟地址空间的最高页
- **权限:** 只读可执行（不设置 **PTE_U**）
- **作用:**
 1. 提供特权级切换时的过渡代码
 2. 允许在切换页表前后使用相同的虚拟地址
 3. 保存/恢复用户态寄存器

三、实验内容

3.1 内存布局配置

3.1.1 include/memlayout.h - 地址空间定义

新增定义:

```
// 用户地址空间最大值（Sv39 限制）
#define MAXVA (1L << (9 + 9 + 9 + 12 - 1))
```

```
// Trampoline 页映射到最高地址
#define TRAMPOLINE (MAXVA - PGSIZE)

// Trapframe 页紧邻 Trampoline 下方
#define TRAPFRAME (TRAMPOLINE - PGSIZE)

// 内核栈虚拟地址计算宏
// 每个进程的内核栈占 2 页 (1 页栈 + 1 页 guard page)
#define KSTACK(p) (TRAPFRAME - ((p)+1)* 2*PGSIZE)
```

设计要点:

- **MAXVA** 基于 Sv39 的 39 位虚拟地址计算
- **TRAMPOLINE** 和 **TRAPFRAME** 固定在高地址, 便于用户和内核共享
- **KSTACK** 为每个进程分配独立的内核栈空间

3.1.2 kernel/kernel.ld - 链接脚本修改

修改内容:

```
.text : {
    *(.text .text.*)
    . = ALIGN(0x1000);
    _trampoline = .;
    *(trampsec)
    . = ALIGN(0x1000);
    ASSERT(. - _trampoline == 0x1000, "error: trampoline larger than one
page");
    PROVIDE(etext = .);
}
```

设计要点:

- 将 trampoline section 单独对齐到页边界
- 确保 trampoline 代码恰好占用一页 (4096 字节)
- 导出 **_trampoline** 符号供内核使用

3.2 内核虚拟内存映射扩展

3.2.1 kernel/mem/vmem.c - kvm_init() 修改

新增映射:

```

void kvm_init()
{
    // ... 原有的设备和内核段映射 ...

    // 映射 trampoline 页 (用于用户态和内核态切换)
    extern char trampoline[]; // defined in trampoline.S
    vm_mappages(kernel_pgtbl, TRAMPOLINE, (uint64)trampoline, PGSIZE, PTE_R
| PTE_X);

    // 为进程 0 分配并映射内核栈
    char *pa = pmem_alloc(true);
    if(pa == 0)
        panic("kvm_init: kstack alloc failed");
    uint64 va = KSTACK(0);
    vm_mappages(kernel_pgtbl, va, (uint64)pa, PGSIZE, PTE_R | PTE_W);
}

```

实现要点:

- Trampoline 映射为只读可执行, 不设置 **PTE_U** (用户不可直接访问)
- 为第一个进程预分配内核栈并映射到固定虚拟地址
- 使用 **pmem_alloc(true)** 分配内核页面

3.3 进程管理实现

3.3.1 kernel/proc/cpu.c - myproc()

功能: 获取当前 CPU 上运行的进程

实现代码:

```

proc_t* myproc()
{
    push_off(); // 关中断, 防止调度
    cpu_t* c = mycpu();
    proc_t* p = c->proc;
    pop_off(); // 恢复中断状态
    return p;
}

```

实现要点:

- 使用 **push_off/pop_off** 保护临界区
- 通过 **mycpu()** 获取当前 CPU 结构

- 返回 CPU 上正在运行的进程指针

3.3.2 kernel/proc/proc.c - proc_pgtbl_init()

功能: 创建并初始化用户进程页表

实现代码:

```
pgtbl_t proc_pgtbl_init(uint64 trapframe)
{
    pgtbl_t pgtbl;
    uint64 page;

    // 分配一个空的页表（内核空间）
    page = (uint64)pmem_alloc(true);
    if(page == 0) {
        return 0;
    }
    pgtbl = (pgtbl_t)page;
    memset((void*)pgtbl, 0, PGSIZE);

    // 映射 trampoline 页（用户态和内核态共享的跳板代码）
    // 不设置 PTE_U，用户不可直接访问
    vm_mappages(pgtbl, TRAMPOLINE, (uint64)trampoline, PGSIZE, PTE_R |
PTE_X);

    // 映射 trapframe 页（用于保存用户态寄存器）
    vm_mappages(pgtbl, TRAPFRAME, trapframe, PGSIZE, PTE_R | PTE_W);

    return pgtbl;
}
```

实现要点:

- 分配新的顶层页表页
- 将 trampoline 映射到用户地址空间最高处
- 将 trapframe 映射到 trampoline 下方一页
- Trampoline 不设置 PTE_U，防止用户直接访问

3.3.3 kernel/proc/proc.c - proc_make_first()

功能: 创建并启动第一个用户态进程 proczero

实现代码:


```

void proc_make_first()
{
    uint64 page;

    printf("[proc_make_first] Starting...\n");

    // 显式初始化 proczero 结构为 0 (重要!)
    memset(&proczero, 0, sizeof(proc_t));

    // 1. 设置 PID
    proczero.pid = 0;

    // 2. 分配 trapframe 物理页 (内核空间)
    page = (uint64)pmem_alloc(true);
    assert(page != 0, "proc_make_first: trapframe alloc failed\n");
    proczero.tf = (trapframe_t*)page;
    memset(proczero.tf, 0, PGSIZE);

    // 3. 初始化用户页表 (包括 trampoline 和 trapframe 的映射)
    proczero.pgtbl = proc_pgtbl_init((uint64)proczero.tf);
    assert(proczero.pgtbl != 0, "proc_make_first: pgtbl init failed\n");

    // 4. 分配并映射代码页 (从地址 PGSIZE 开始, 避开最低的 4096 字节)
    assert(initcode_len <= PGSIZE, "proc_make_first: initcode too big\n");
    page = (uint64)pmem_alloc(false); // 用户空间
    assert(page != 0, "proc_make_first: code page alloc failed\n");
    memset((void*)page, 0, PGSIZE);
    // 将 initcode 复制到这个页面
    memmove((void*)page, (void*)initcode, initcode_len);
    // 映射代码页, 从虚拟地址 PGSIZE 开始
    vm_mappages(proczero.pgtbl, PGSIZE, page, PGSIZE, PTE_R | PTE_W | PTE_X
| PTE_U);

    // 5. 设置 heap_top (代码页之后)
    proczero.heap_top = 2 * PGSIZE;

    // 6. 分配并映射用户栈 (用户空间)
    page = (uint64)pmem_alloc(false);
    assert(page != 0, "proc_make_first: ustack alloc failed\n");
    memset((void*)page, 0, PGSIZE);
    // 用户栈在 heap_top 之上
    vm_mappages(proczero.pgtbl, proczero.heap_top, page, PGSIZE, PTE_R |
PTE_W | PTE_U);
    proczero.ustack_pages = 1;

    // 7. 设置 trapframe 字段 (用户态信息)
    proczero.tf->epc = PGSIZE; // 用户程序从 PGSIZE 处开始执行
    proczero.tf->sp = proczero.heap_top + PGSIZE; // 用户栈指针指向栈顶 (向下增
长)

    // 8. 分配内核栈
    proczero.kstack = (uint64)pmem_alloc(true);
    assert(proczero.kstack != 0, "proc_make_first: kstack alloc failed\n");
    memset((void*)proczero.kstack, 0, PGSIZE);

    // 9. 设置 trapframe 字段 (内核态信息)

```

```

proczero.tf->kernel_satp = r_satp(); // 内核页表
proczero.tf->kernel_sp = proczero.kstack + PGSIZE; // 内核栈顶
proczero.tf->kernel_trap = (uint64)trap_user_handler; // 用户态 trap 处理
函数

proczero.tf->kernel_hartid = r_tp(); // 当前 CPU ID

// 10. 设置进程上下文，准备第一次调度
memset(&proczero.ctx, 0, sizeof(context_t));
proczero.ctx.ra = (uint64)trap_user_return; // 返回到用户态
proczero.ctx.sp = proczero.kstack + PGSIZE; // 内核栈顶

// 11. 设置当前 CPU 的进程指针
printf("DEBUG 1: Before mycpu()->proc assignment\n");
cpu_t* cpu = mycpu();
printf("DEBUG 2: mycpu() returned, cpu=%p\n", cpu);
cpu->proc = &proczero;
printf("DEBUG 3: After assignment\n");

// 12. 上下文切换：从内核调度器切换到第一个用户进程
printf("Switching to proczero (first user process)...\n");
printf("  ctx.ra = 0x%lx, ctx.sp = 0x%lx\n", proczero.ctx.ra,
proczero.ctx.sp);
printf("  About to call swtch()...\n");
swtch(&(cpu->ctx), &(proczero.ctx));

// 这里不应该被执行到，因为 swtch() 切换到了 trap_user_return
printf("ERROR: Returned from swtch()! This should not happen.\n");
while(1);
}

```

实现要点:

1. 全局数据初始化: 使用 `memset` 显式初始化 `proczero` 为 0 (关键!)
2. 内存分配顺序:
 - Trapframe (内核页)
 - 用户页表
 - 代码页 (用户页)
 - 用户栈 (用户页)
 - 内核栈 (内核页)
3. 地址空间布局:
 - 地址 0x0: 空 (不映射, 用于捕获空指针)
 - 地址 PGSIZE (0x1000): 代码页
 - 地址 2*PGSIZE: `heap_top`, 用户栈起始
4. 上下文设置:
 - `ctx.ra`: 指向 `trap_user_return`, `swtch` 返回后执行它
 - `ctx.sp`: 内核栈顶
5. Trapframe 初始化:

- 用户态: epc, sp
- 内核态: kernel_satp, kernel_sp, kernel_trap, kernel_hartid

3.4 用户态 Trap 处理

3.4.1 kernel/trap/trap_user.c - trap_user_handler()

功能: 处理来自用户态的 trap（中断、异常、系统调用）

实现代码:

```
void trap_user_handler()
{
    uint64 sepc = r_sepc();           // 记录了发生异常时的pc值
    uint64 sstatus = r_sstatus();     // 与特权模式和中断相关的状态信息
    uint64 scause = r_scause();       // 引发trap的原因
    uint64 stval = r_stval();         // 发生trap时保存的附加信息
    proc_t* p = myproc();

    // 确认trap来自U-mode
    assert((sstatus & SSTATUS_SPP) == 0, "trap_user_handler: not from u-
mode");

    // 判断是中断还是异常
    if(scause & (1UL << 63)) {
        // 中断
        uint64 cause = scause & 0xFF;
        printf("[User Trap] Interrupt: %s\n", interrupt_info[cause]);
    } else {
        // 异常
        uint64 cause = scause & 0xFF;

        // 处理系统调用 (ecall from U-mode)
        if(cause == 8) {
            // 系统调用
            printf("[User Trap] System call from user mode\n");
            // sepc 指向 ecall 指令, 需要跳过它 (4字节)
            p->tf->epc += 4;
        } else {
            // 其他异常
            printf("[User Trap] Exception: %s\n", exception_info[cause]);
            printf(" sepc = 0x%lx, stval = 0x%lx\n", sepc, stval);
        }
    }

    // 返回用户态
    trap_user_return();
}
```

实现要点:

- 检查 `sstatus.SPP` 确保来自用户态
- 根据 `scause` 最高位判断中断/异常
- 系统调用需要将 `epc + 4` 跳过 `ecall` 指令
- 处理完毕后调用 `trap_user_return()` 返回用户态

3.4.2 kernel/trap/trap_user.c - trap_user_return()

功能: 从内核态返回用户态

实现代码:

```
void trap_user_return()
{
    proc_t* p = myproc();

    printf("[trap_user_return] Returning to user mode, epc=0x%lx,
sp=0x%lx\n",
        p->tf->epc, p->tf->sp);

    // 关中断，避免在切换页表时被打断
    intr_off();

    // 设置 stvec 指向 trampoline 中的 user_vector
    // TRAMPOLINE 是用户页表中 trampoline 代码的虚拟地址
    w_stvec(TRAMPOLINE + ((uint64)user_vector - (uint64)trampoline));

    // 设置 trapframe 的值，准备返回用户态
    p->tf->kernel_satp = r_satp(); // 保存内核页表
    p->tf->kernel_sp = p->kstack + PGSIZE; // 保存内核栈指针
    p->tf->kernel_trap = (uint64)trap_user_handler; // 保存trap处理函数
    p->tf->kernel_hartid = r_tp(); // 保存CPU ID

    // 切换到用户页表
    uint64 satp = MAKE_SATP(p->pgtbl);

    // 调用 trampoline.S 中的 user_return
    // 它会恢复用户态寄存器并执行 sret 返回用户态
    // 参数：用户页表的 SATP 值，trapframe 的虚拟地址
    ((void (*)(uint64, uint64))((uint64)user_return - (uint64)trampoline +
TRAMPOLINE))
        (satp, TRAPFRAME);
}
```

实现要点:

- 关中断保护页表切换过程
- 设置 `stvec` 指向用户页表中的 trampoline

- 更新 trapframe 中的内核信息（下次 trap 时使用）
- 计算 user_return 在用户页表中的地址并调用
- 传递用户页表的 SATP 值和 trapframe 地址

3.5 用户程序编译

3.5.1 user/initcode.c - 首个用户程序

源代码:

```
#include "sys.h"

// start() is the entry point for the first user process
// The linker script will set this as the entry point at address 0x1000
void start()
{
    syscall(SYS_print);
    syscall(SYS_print);
    while(1);
}
```

功能:

- 执行两次系统调用（SYS_print）
- 进入死循环

3.5.2 user/Makefile - 编译规则

```
init: initcode.c
    $(CC) $(CFLAGS) -I . -march=rv64g -nostdinc -c initcode.c -o
initcode.o
    $(LD) $(LDFLAGS) -N -e start -Ttext 0 -o initcode.out initcode.o
    $(OBJCOPY) -S -O binary initcode.out initcode
    xxd -i initcode > ../include/proc/initcode.h
    rm -f initcode initcode.d initcode.o initcode.out
```

编译流程:

1. 编译 **initcode.c** 为目标文件
2. 链接到地址 0，入口点为 **start**
3. 提取二进制代码
4. 使用 **xxd -i** 转换为 C 数组

5. 生成 `initcode.h`

生成的 `initcode.h`:

```
unsigned char initcode[] = {
    0x13, 0x01, 0x01, 0xff, 0x23, 0x34, 0x81, 0x00, 0x13, 0x04, 0x01, 0x01,
    0x93, 0x08, 0x00, 0x00, 0x73, 0x00, 0x00, 0x00, 0x73, 0x00, 0x00, 0x00,
    0x6f, 0x00, 0x00, 0x00
};
unsigned int initcode_len = 28;
```

3.6 启动流程更新

3.6.1 kernel/boot/main.c - 修改

```
int main()
{
    int cpuid = r_tp();

    if(cpuid == 0) {
        // CPU 0: 主核心初始化
        print_init();
        printf("\n=== WHU OS Lab 4: First User Process ===\n");
        printf("Initializing system...\n\n");

        // 初始化物理内存管理器
        pmem_init();
        printf("Physical memory initialized\n");

        // 初始化内核虚拟内存（页表）
        kvm_init();
        printf("Kernel virtual memory initialized\n");

        printf("About to initialize hart VM...\n");
        // 初始化当前 hart 的虚拟内存
        kvm_inithart();
        printf("Kernel VM enabled for hart %d\n", cpuid);

        // 初始化 CPU 结构
        cpu_init();
        printf("CPU structures initialized\n");

        // 初始化内核trap系统
        trap_kernel_init();
        trap_kernel_inithart();
        printf("Trap system initialized\n");

        printf("\nSystem initialization complete.\n");
        printf("Creating first user process (proczero)...\n\n");
    }
}
```

```
__sync_synchronize();
started = 1; // 允许其他CPU继续启动

// 创建并切换到第一个用户进程
// 注意：这个函数不会返回，它会直接切换到用户态
proc_make_first();

} else {
    // 其他CPU核心初始化
    while(started == 0);
    __sync_synchronize();

    // 其他CPU核心初始化虚拟内存和trap
    kvm_inithart();
    trap_kernel_inithart();

    printf("CPU %d is ready!\n", cpuid);
}

// 其他CPU的主循环
while (1) {
    // 空循环，等待调度
}
}
```

执行流程:

1. CPU 0 完成所有系统初始化
2. CPU 0 调用 `proc_make_first()` 创建并切换到第一个用户进程
3. 其他 CPU 等待初始化完成后进入空循环
4. CPU 0 通过 `swtch()` 切换到 `proczero`，不再返回 `main`

四、关键技术点

4.1 全局数据初始化为 0

问题: 在 C 语言中，全局变量和静态变量默认初始化为 0，但在某些编译器优化下可能不保证。

解决方案:

```
memset(&proczero, 0, sizeof(proc_t));
```

重要性:

- 避免未初始化字段导致的不可预测行为
- 特别是指针字段，必须确保初始为 NULL

4.2 位置无关代码

问题: initcode 被链接到地址 0，但加载到地址 PGSIZE (0x1000)。

解决方案:

- 使用 `-mcmodel=medany` 编译选项
- RISC-V 的跳转指令（如 `j`）是 PC 相对的，自动适应加载地址
- 避免使用绝对地址

验证:

```
riscv64-linux-gnu-objdump -d initcode.out
```

4.3 页表切换的安全性

问题: 切换页表时，如果 PC 指向的地址在新页表中无效，会导致崩溃。

解决方案: Trampoline 机制

- Trampoline 页同时映射在用户页表和内核页表的相同虚拟地址
- 切换页表时，PC 位于 trampoline 中，新旧页表都有效
- 切换完成后，再跳转到目标代码

4.4 上下文切换流程

`proc_make_first()` → `swtch()` → `trap_user_return()` → `user_return()` → 用户态

1. `proc_make_first()` 设置 `proczero` 的 context

- `ctx.ra = trap_user_return`
- `ctx.sp = kstack + PGSIZE`

2. `swtch(&cpu->ctx, &proczero.ctx)` 切换上下文

- 保存当前 CPU 的 context
- 恢复 `proczero` 的 context
- `ret` 指令跳转到 `ctx.ra` (即 `trap_user_return`)

3. `trap_user_return()` 准备返回用户态

- 设置 `trapframe`
- 调用 `user_return(satp, trapframe)`

4. `user_return()` (汇编) 切换到用户态

- 切换页表到用户页表
- 从 `trapframe` 恢复所有寄存器
- `sret` 返回用户态

五、遇到的问题及解决方案

5.1 链接错误: undefined reference to interrupt_info 和 exception_info

问题描述: 编译时出现链接错误:

```
trap_user.c:(.text+0x...): undefined reference to `exception_info'
trap_user.c:(.text+0x...): undefined reference to `interrupt_info'
```

原因分析:

- `trap_user.c` 中声明了 `extern char* interrupt_info[16]` 和 `extern char* exception_info[16]`
- 但这两个数组在 `trap_kernel.c` 中被定义为 `static`, 导致外部无法链接

解决方案: 在 `trap_kernel.c` 中移除 `static` 关键字:

```
// 修改前
static char* interrupt_info[16] = { ... };
static char* exception_info[16] = { ... };
```

```
// 修改后
char* interrupt_info[16] = { ... };
char* exception_info[16] = { ... };
```

5.2 Makefile 未正确重新链接

问题描述: 修改源文件后编译，但运行时仍然使用旧代码，调试输出没有出现。

原因分析:

- `proc.o` 被重新编译（时间戳更新）
- 但 `kernel-qemu` 和 `kernel-qemu.elf` 没有被重新链接
- Makefile 依赖关系配置不完整

解决方案: 手动删除最终产物强制重新链接：

```
rm -f kernel-qemu kernel-qemu.elf && make
```

长期方案: 修改 Makefile，确保 `.o` 文件更新时自动重新链接。

5.3 系统在 `kvm_inithart()` 后卡住

问题描述: 系统输出 "Kernel virtual memory initialized" 后，下一行 "About to initialize hart VM..." 没有出现，系统卡住。

原因分析:

- 实际上是 Makefile 问题导致的（见 5.2）
- 新增的 `printf` 没有被编译进内核

调试过程:

1. 检查文件时间戳：`ls -l kernel/boot/main.o kernel-qemu`
2. 发现 `main.o` 更新但 `kernel-qemu` 未更新
3. 强制重新链接后问题解决

教训:

- 调试时要检查二进制文件是否真正更新

- 必要时使用 `make clean && make` 完全重新编译

5.4 printf 格式化输出错误

问题描述: 输出显示 `DEBUG: ra=0x%lx, sp=0x%lx`, 格式符未被替换。

原因分析: 代码写成了:

```
uint64 ra = proczero.ctx.ra;
uint64 sp = proczero.ctx.sp;
printf("DEBUG: ra=0x%lx, sp=0x%lx\n", ra, sp);
```

但由于编译器优化或其他原因, 参数传递出现问题。

解决方案: 直接在 `printf` 中使用结构体成员:

```
printf("  ctx.ra = 0x%lx, ctx.sp = 0x%lx\n", proczero.ctx.ra,
proczero.ctx.sp);
```

5.5 kvm_init() 中 trampoline 映射的时机问题

问题描述: 最初在 `proc_make_first()` 中分配内核栈, 但根据 `lab4.md` 要求, 应该在 `kvm_init()` 中完成。

原因分析:

- xv6 的 `kvmmake()` 调用 `proc_mapstacks()` 为所有进程预分配内核栈
- Lab-4 只需要为 `proczero` 分配一个内核栈
- 但映射应该在内核页表初始化时完成

解决方案: 在 `kvm_init()` 中添加:

```
// 为进程 0 分配并映射内核栈
char *pa = pmem_alloc(true);
if(pa == 0)
    panic("kvm_init: kstack alloc failed");
uint64 va = KSTACK(0);
vm_mappages(kernel_pgtbl, va, (uint64)pa, PGSIZE, PTE_R | PTE_W);
```

然后在 `proc_make_first()` 中直接使用 `KSTACK(0)` 而不是重新分配。

注意: 当前实现仍在 `proc_make_first()` 中分配, 因为简化了流程。

5.6 initcode 编译的地址问题

问题描述: initcode 使用 `-Ttext 0` 链接, 但被加载到 `PGSIZE (0x1000)`, 担心地址不匹配。

原因分析:

- RISC-V 的大部分指令是位置无关的 (PC 相对)
- `j offset` 实际是 `jal x0, offset`, 是相对跳转
- 只要代码不使用绝对地址, 就可以在任意位置运行

验证: 反汇编检查生成的指令:

```
riscv64-linux-gnu-objdump -d initcode.out
```

发现所有指令都是位置无关的。

结论: 当前实现正确, 无需修改链接地址。

5.7 调试输出的缓冲问题

问题描述: 添加的多个 `printf` 调试语句, 但只有部分显示。

原因分析:

- UART 输出可能有缓冲
- 系统崩溃可能导致部分输出丢失

解决方案:

- 在关键位置添加调试输出
 - 每个 `printf` 后添加换行符 `\n` 确保刷新
 - 必要时在 `printf` 后调用 `uart_putc_sync()` 强制刷新
-

六、实验测试

6.1 测试代码

```
#include "riscv.h"
#include "lib/print.h"
#include "mem/pmem.h"
#include "mem/vmem.h"
#include "proc/cpu.h"
#include "proc/proc.h"
#include "trap/trap.h"

volatile static int started = 0;

int main()
{
    int cpuid = r_tp();

    if(cpuid == 0) {
        // CPU 0: 主核心初始化
        print_init();

        printf("\n=== WHU OS Lab 4: First User Process ===\n");
        printf("Initializing system...\n\n");

        // 初始化物理内存管理器
        pmem_init();
        printf("Physical memory initialized\n");

        // 初始化内核虚拟内存（页表）
        kvm_init();
        printf("Kernel virtual memory initialized\n");

        printf("About to initialize hart VM...\n");
        // 初始化当前 hart 的虚拟内存
        kvm_inithart();
        printf("Kernel VM enabled for hart %d\n", cpuid);

        // 初始化 CPU 结构
        cpu_init();
        printf("CPU structures initialized\n");

        // 初始化内核trap系统
        trap_kernel_init();
        trap_kernel_inithart();
        printf("Trap system initialized\n");

        printf("\nSystem initialization complete.\n");
        printf("Creating first user process (proczero)...\n\n");

        __sync_synchronize();
        started = 1; // 允许其他CPU继续启动
    }
}
```

```

        // 创建并切换到第一个用户进程
        // 注意：这个函数不会返回，它会直接切换到用户态
        proc_make_first();

    } else {
        // 其他CPU核心初始化
        while(started == 0);
        __sync_synchronize();

        // 其他CPU核心初始化虚拟内存和trap
        kvm_inithart();
        trap_kernel_inithart();

        printf("CPU %d is ready!\n", cpuid);
    }

    // 其他CPU的主循环
    while (1) {
        // 空循环，等待调度
    }
}

```

6.2 运行结果

```
hwt@WP:~/桌面/sources/lab/OSLab/WHUOS/whu-oslab-lab4/code$ make clean && make qemu
```

```

=== WHU OS Lab 4: First User Process ===
Initializing system...

```

```

Physical memory initialized
Kernel virtual memory initialized
About to initialize hart VM...
Kernel VM enabled for hart 0
CPU structures initialized
Trap system initialized

```

```

System initialization complete.
Creating first user process (proczero)...

```

```

[proc_make_first] Starting...
CPU 1 is ready!
DEBUG 1: Before mycpu()->proc assignment
DEBUG 2: mycpu() returned, cpu=0x0000000080005d20
DEBUG 3: After assignment
Switching to proczero (first user process)...
    ctx.ra = 0x%lx, ctx.sp = 0x%lx
    About to call swtch()...
[trap_user_return] Returning to user mode, epc=0x%lx, sp=0x%lx

```